

Project Proposal: Design and Implementation of a Micro-segmentation based Zero Trust Architecture (ZTA) Platform

Student Name: Minsoo Ahn

Date: August 2025

Topic: Implementation of a dynamic Zero Trust access control platform for cloud-native environments.

1. Project Overview: Scope and Significance

Traditional perimeter-based security models (Castle-and-Moat) are no longer sufficient due to the rise of cloud computing. This project moves beyond theoretical study to **design, implement, and validate** a functional Zero Trust Architecture (ZTA) platform.

1.1 Final Deliverable: The "Zero Trust Sandbox"

The ultimate output of this project is a **self-contained Kubernetes-based security testbed**. This is not just a software installation, but a fully configured environment containing:

1. **Victim Application:** A microservices-based web application designed to simulate a real-world target.
2. **Defense Layer (The ZTA Platform):** An integrated control plane (using Istio & OPA) that enforces security policies.
3. **Attack Simulations:** A set of scripts that attempt to breach the system, serving as proof of the platform's effectiveness.

1.2 Key Functions: What does it actually do?

Unlike a standard firewall that sits at the network edge, this platform functions as a **ubiquitous internal guard**:

- **Traffic Interception:** It intercepts 100% of the traffic *between* internal services (East-West traffic), which is invisible to traditional firewalls.
- **Dynamic Evaluation:** It checks **Identity** (Who?), **Context** (Where/When?), and **Device Posture** (Is the device healthy/patched?) for every single request.
- **Micro-segmentation:** It isolates a compromised service immediately, preventing the attacker from jumping to other servers.

1.3 Relevance to ZTA: Why build this?

Zero Trust is often discussed as an abstract concept ("Never Trust, Always Verify"). This

project translates that abstract concept into **concrete engineering**:

- **Implementation of NIST Standards:** It directly maps the logical components of NIST SP 800-207 (Policy Decision Point, Policy Enforcement Point) to actual software (OPA, Istio).
- **Proof of Concept:** It demonstrates *how* to remove implicit trust from a network, showing that ZTA is a viable engineering solution, not just a policy.

1.4 Project Benefits: What will be gained?

By building this platform, the project aims to achieve:

1. **Verification of Theory:** Empirically proving that ZTA can stop lateral movement where traditional models fail.
2. **Practical Engineering Skills:** Gaining hands-on expertise with industry-standard Cloud-Native tools (Kubernetes, Service Mesh).
3. **Bridging the Gap:** Understanding the complexity of implementing security policies in a live, distributed system.

2. Problem Statement & Scenarios

This project focuses on mitigating the following specific security scenarios by implementing the ZTA prototype:

Scenario A: Prevention of Lateral Movement

- **Problem:** An attacker compromises a frontend web server. In a flat network, they can immediately scan and access the backend database.
- **Goal:** The prototype should deny access to the database unless specific, dynamic criteria are met, verifying that the implementation successfully segments the network.

Scenario B: Context & Device Posture Access Control

- **Problem:** A valid user's credentials are stolen, or a legitimate user attempts access from a **compromised/outdated device** (e.g., OS vulnerabilities, disabled firewall).
- **Goal:** The platform should detect **device health status** and context changes (location, time), enforcing policy blocks if the device is deemed insecure, even with valid credentials.

3. Implementation Approach & Tools

To solve the problems above, the platform will be constructed using a **Service Mesh** and **Policy as Code** approach.

Note: The following software components are selected as tools to implement and evaluate the proposed ZTA platform. These are not contributions in themselves but serve as practical means to produce the prototype and observe system behavior.

3.1 Architecture Overview

1. **Policy Decision Point (PDP):** The "brain" (OPA) that evaluates requests against policies.
2. **Policy Enforcement Point (PEP):** The "gatekeeper" (Istio/Envoy) that intercepts traffic.
3. **Identity Provider (IdP):** The authentication authority (Keycloak).

3.2 Technology Stack

- **Infrastructure:** Kubernetes (K8s) or Minikube.
- **Service Mesh:** Istio (uses Envoy Proxy) for traffic interception and mTLS.
- **Policy Engine: OPA (Open Policy Agent)** for defining access rules in Rego.
- **Identity Management: Keycloak** for OIDC/JWT management.
- **VPN/Tunneling (Baseline): WireGuard** and **OpenVPN** may be used to establish secure remote access baselines for comparison.

4. 17-Week Project Roadmap

This schedule outlines the development and testing phases over the semester.

Phase 1: Research & Design (Weeks 1-4)

- **Week 1:** Literature review of NIST SP 800-207.
- **Week 2:** Requirement analysis and defining the microservices architecture.
- **Week 3:** Architectural Design: Interaction between Istio, OPA, and the App.
- **Week 4:** Environment Setup: Local Kubernetes (Minikube) and Git repository.

Phase 2: Core Infrastructure Implementation (Weeks 5-8)

- **Week 5:** Deploying sample microservices (Frontend, Backend, DB).
- **Week 6:** Implementing **Keycloak** for Identity Management.
- **Week 7:** Integrating **Istio Service Mesh** and injecting sidecars.
- **Week 8:** Establishing mTLS between services.

Phase 3: ZTA Logic & Policy Development (Weeks 9-12)

- **Week 9:** Integrating **OPA** with Istio as an external authorization provider.
- **Week 10:** Writing Rego policies for Scenario A (Service-to-Service).
- **Week 11:** Writing Rego policies for **Scenario B (Device Posture & Context)**.
- **Week 12:** Developing a simple dashboard to visualize logs and decisions.

Phase 4: Testing & Simulation (Weeks 13-15)

- **Week 13: Simulation:** Lateral movement attack attempts.
- **Week 14: Simulation:** Context-aware access tests (invalid IP/Time/Insecure Device).
- **Week 15:** Performance testing (latency overhead).

Phase 5: Conclusion (Weeks 16-17)

- **Week 16:** Analyzing results and documenting findings.

- **Week 17:** Final Report writing and project presentation.

5. Expected Outcomes

By the end of this project, we will have:

1. **A working Zero Trust Architecture platform prototype** deployed in a local Kubernetes environment.
2. **Evidence** of how the platform handles lateral movement and context-based access decisions (success/fail logs).
3. **Observations** on implementation challenges and performance characteristics compared to baseline methods.

6. Appendix – Tools References and Descriptions

This appendix provides links and brief descriptions of the practical tools used in this project implementation.

NIST SP 800-207 – Zero Trust Architecture Standard

- **Why included:** Defines the core concepts and logical components of Zero Trust Architecture (ZTA).
- **Link:** [NIST SP 800-207 PDF](#)

WireGuard – Kernel-level VPN implementation

- **Why included:** Practical example of a low-level tunneling tool, potentially used for secure ingress or baseline comparison.
- **Link:** [NDSS 2017 Programme - WireGuard](#)

Open Policy Agent (OPA) – Policy-as-code engine

- **Why included:** Essential tool for implementing dynamic Policy Decision Point (PDP) logic.
- **Link:** [OPA Documentation](#)

OpenVPN – User-space VPN

- **Why included:** Representative user-space VPN implementation used as a baseline for traditional access control comparison.
- **Link:** [OpenVPN Community](#)

Keycloak – Identity Provider (IdP)

- **Why included:** Off-the-shelf identity management tool used for implementing authentication flows (OIDC/OAuth2).
- **Link:** [About Keycloak - Features & Overview](#)

Istio – Service Mesh

- **Why included:** Industry-standard tool for implementing the Policy Enforcement Point (PEP) and mTLS in Kubernetes.

- **Link:** [Istio Concept - What is Istio?](#)